UNIVERSITÀ DI PISA

ADVANCED SOFTWARE ENGINEERING (LAB)

ASE Project: La Escoba Card Game

Detailed Microservices Architecture and Implementation Report

Authors:
Elena MARTÍNEZ VAZQUEZ
Mario PEREZ PEREZ
Michele F. P. SAGONE

Emails:
e.martinezvazquez@studenti.unipi.it
m.perezperez1@studenti.unipi.it
m.sagone1@studenti.unipi.it

Academic Year 2025–2026
December 16, 2025

Contents

1 Cards Overview

This project utilizes the traditional 40-card **Spanish Deck (Baraja Española)** to play La Escoba. The Cards Service is responsible for defining and maintaining all card data and values, ensuring every other microservice uses the exact same, standardized deck. This section details the composition of the deck and the critical value mapping used for the core game logic.

1.1 Deck Structure and Composition

The deck is composed of 40 cards, divided equally among four distinct suits.

- **Suits:** Oros (Coins), Copas (Cups), Espadas (Swords), and Bastos (Clubs). Each suit is distinct and important for specific scoring bonuses (e.g., collecting the most Oros).
- **Composition:** Each suit contains cards numbered 1 through 7, plus three face cards: Sota (Jack), Caballo (Knight), and Rey (King). Crucially, the nominal numbers 8 and 9 are traditionally omitted from the Spanish 40-card deck, leading to the necessity of mapping the face cards.

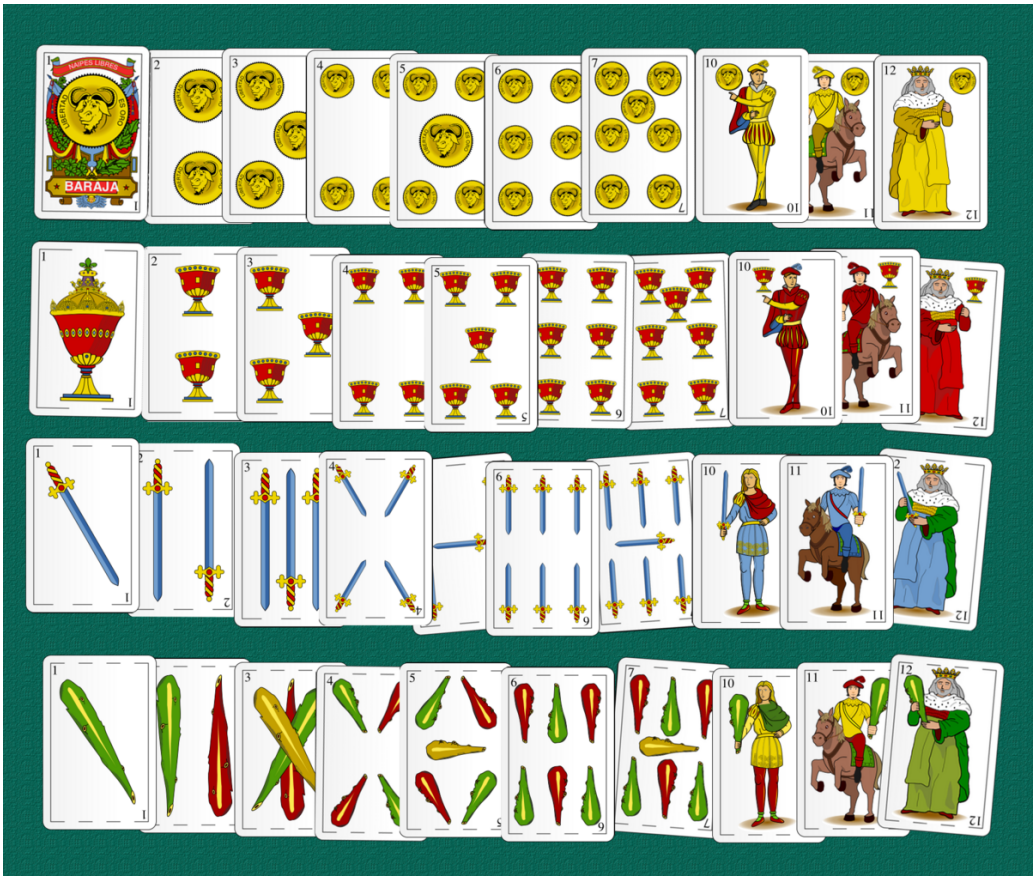


Figure 1: The 40-card Spanish Deck used for La Escoba, showing the four suits and their face cards.

1.2 Card Values and Game Logic Mapping

The central mechanism of La Escoba is the "Capture Rule," which dictates that a played card must combine with one or more cards on the table to sum exactly to **15**. Since the numeric

cards only run up to 7, the face cards must be mapped to higher values (8, 9, and 10) to ensure all cards can participate in capture combinations.

The following table details the necessary value mapping used by the `Match Service` to execute the game logic:

Card Name	Nominal Number	Game Value (Sum to 15)
Numeric Cards (#1–#7)	1–7	1–7
Sota (Jack)	10	**8**
Caballo (Knight)	11	**9**
Rey (King)	12	**10**

Table 1: The critical mapping of face cards to ensure capture combinations resulting in 15 are possible.

1.3 Card Service Role in Architecture

The Cards Service is implemented as a stateless microservice. Its primary function is to serve a fixed JSON array of all 40 cards. This architectural choice provides significant benefits for the system's robustness:

- **Single Source of Truth:** It guarantees that all microservices, especially the Match Service, access the exact same dataset, eliminating potential data inconsistencies or errors in rule implementation.
- **Decoupling:** The core game logic (Match Service) is decoupled from card data definition. Any update to the card values or the addition of new card sets would only require modifying the Cards Service.
- **Data Format:** Each card object includes its unique identifier (used internally by the Match Service for state tracking), its suit, and its pre-calculated `game_value`.

2 Architecture

The system implements a **Microservices Architecture**, adhering to key principles such as Separation of Concerns, Independent Scalability, and Distributed Persistence. The application is composed of six specialized services and two database engines, all containerized and orchestrated via Docker Compose.

2.1 Architecture Diagram

The following diagram illustrates the interaction between the Client (Frontend), the API Gateway, the internal Microservices, and the Data Layer (see Figure **??**).

2.2 Microservices Description

Each microservice is built with Python/Flask and has a single responsibility. The overall structure is illustrated in Figure **??**.

Service	**Tech Stack**	**Responsibility**
`API Gateway`	Flask, Gunicorn	Central proxy. Handles HTTPS termination, routes traffic, and centralizes JWT validation.
`Auth Service`	Flask, PostgreSQL	Authentication provider. Manages registration, bcrypt encryption, profile updates, and JWT issuance.
`Player Service`	Flask, PostgreSQL	Manages user profiles, leaderboards, and the Friend System relationship graph.
`Cards Service`	Flask	Stateless service providing the immutable catalog of the 40-card Spanish deck.
`Match Service`	Flask, Redis	Core game engine. Orchestrates turns, enforces rules, and manages the matchmaking queue.
`Auth Service`	Flask, PostgreSQL	Stores immutable logs of finished matches and provides historical data for analytics.

Table 2: Overview of Microservices and their roles.

2.3 Design Choices: Data and Functionality Splitting

We adopted the **Database-per-Service** pattern conceptually to decouple the services.

- **Persistence (PostgreSQL):** Used for critical, long-term data requiring ACID compliance. The schema is distributed across logical domains: `users` (Auth), `players` and `friends` (Player), and `match_history` (History).
- **High-Speed State (Redis):** Chosen for the `Match Service` to handle volatile data.
  - **Game States:** Stores active match objects (hands, table, scores) for rapid access during gameplay loops.
  - **Matchmaking Queue:** Uses Redis Lists to implement a FIFO queue for automatic player pairing.
  - **TTL:** Automatic expiration cleans up abandoned games without manual intervention.

- **Separation of Player and Auth:** Decoupling authentication (Auth Service) from user activity (Player Service) ensures that high traffic on social features or leaderboards does not impact the stability of the login system.

2.4 Key Usage Flows and Inter-Service Communication

Internal communication is restricted to specific business logic needs to maintain loose coupling:

1. **Create Match:** The `Match Service` initializes the game state in Redis. Upon completion, it does not store the result itself but offloads it to the `History Service`.
2. **Play a Card:** The client communicates with the API Gateway, which proxies to the `Match Service`. The service validates the move against the Redis state, updates the board, and returns the new state.
3. **Match Conclusion:**
   - **Match → History:** To persist the final score and move log permanently.
   - **Match → Player:** To update the user's global statistics (wins/losses) for the leaderboard.
4. **Friend System:** Handled entirely within the `Player Service` and its PostgreSQL tables, allowing users to send requests and view online status without querying the Auth Service.

2.5 Why this Stack?

- **Microservices:** Allows selective scalability (e.g., replicating the Match Service under high load) and fault tolerance.
- **Flask:** Lightweight, easy to deploy in Docker, and provides native HTTPS support essential for the project's security requirements.
- **Redis:** Chosen over a SQL database for game state because of its in-memory speed and support for complex data structures needed for real-time turn management.

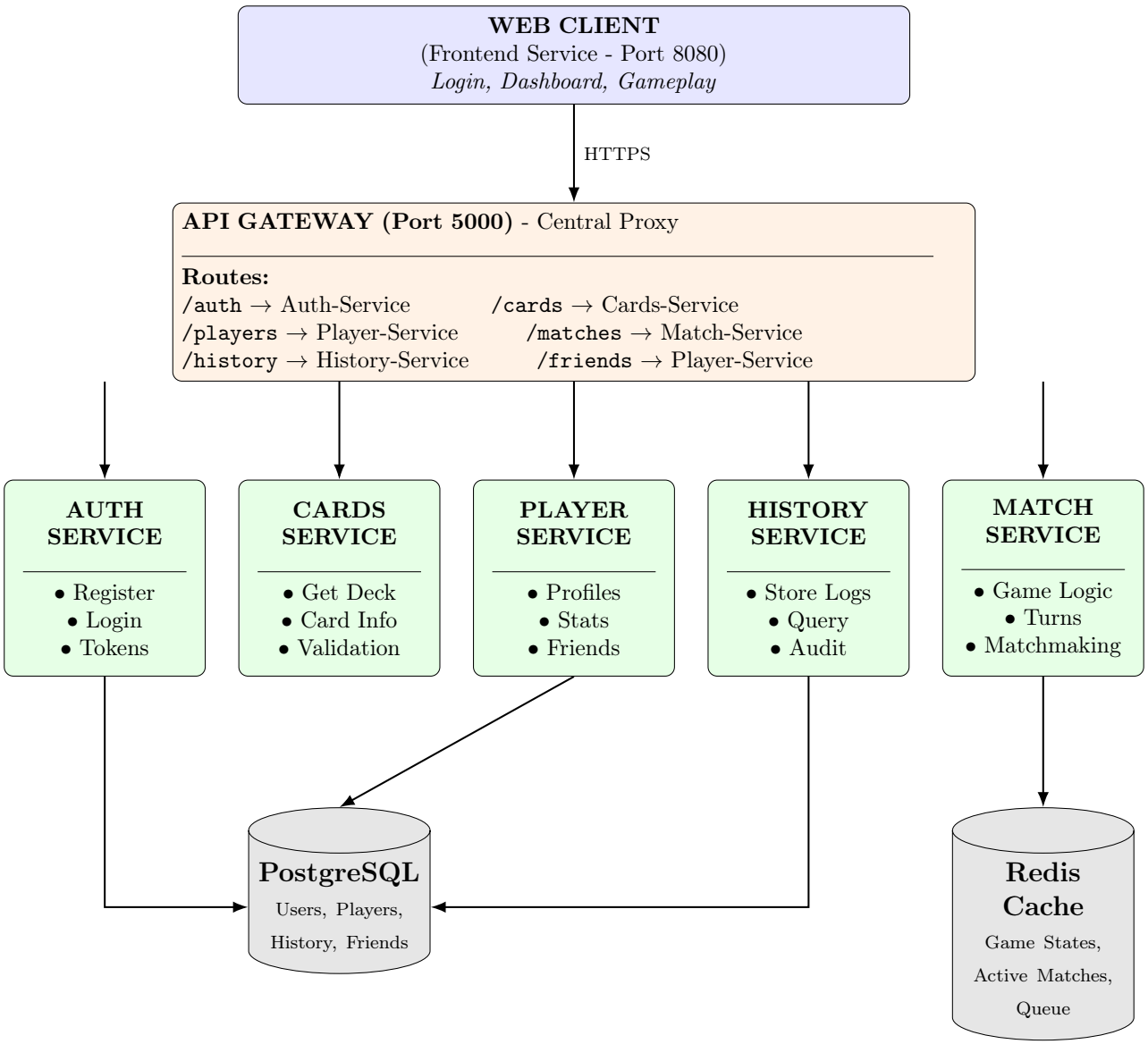


Figure 2: Detailed Data Flow and Routing Diagram.

3 User Stories

This section maps the functional requirements (User Stories) defined in the project specifications to the actual API implementation. The stories are categorized by priority color, mapping each requirement to the specific endpoint and microservice responsible for its realization.

3.1 Mandatory Requirements (Red & Yellow Stories)

These stories represent the core functionality of the system: account management, card visibility, gameplay mechanics, and history tracking.

User Story (Red & Yellow)	**Endpoint(s)**	**Service**
Create an account	`POST /auth/register`	Auth
Login into the game	`POST /auth/login`	Auth
Check/modify my profile	`GET /auth/me`, `PUT /auth/update`	Auth
Be safe about my account data	(Implemented via HTTPS & JWT)	Gateway
See the overall card collection	`GET /cards/cards`	Cards
View the details of a card	`GET /cards/cards/{id}`	Cards
Start a new game	`POST /matches`	Match
Select the subset of cards	(Internal: Automatic Deck Dealing)	Match
Select a card (Play turn)	`POST /matches/{id}/play`	Match
Know the score	`GET /matches/{id}`	Match
See the score	`GET /matches/{id}`	Match
Know the turns (Round info)	`GET /matches/{id}`	Match
Know who won the turn	Response of `/play` (captured cards)	Match
Know who won a match	`GET /matches/{id}` (status)	Match
Ensure rules are not violated	(Internal: `play` validation logic)	Match
View list of my old matches	`GET /history/{username}`	History
View details of a match	`GET /history/match/{id}`	History
View the leaderboards	`GET /players/leaderboard/top`	Player
Prevent tampering old matches	(Read-only API Architecture)	History

Table 3: Mapping of Red and Yellow User Stories to API Endpoints.

3.2 Additional Features (Green Stories)

These stories represent extended functionalities implemented to enhance the user experience, such as social interactions, bot integration, and real-time reactions.

User Story (Green)	**Endpoint(s)**	**Service**
Who's turn it is	`GET /matches/{id}` (field: turn)	Match
See which cards I have	`GET /matches/{id}` (field: hand)	Match
To be able to see the cards in my hand	`GET /matches/{id}`	Match
Play against a bot	`POST /matches` (player2="CPU")	Match
Able to surrender	`POST /matches/{id}/surrender`	Match
Send a battle invitation	`POST /invites`	Match
Request a rematch	(UI triggers `POST /matches`)	Match
Play locally (Guest Mode)	`POST /matches` (player2="Guest")	Match
Use emojis/reactions	`POST /matches/{id}/react`	Match
Add friends	`POST /friends/request`	Player
View my friends list	`GET /friends/list/{user}`	Player
Remove a friend	`POST /friends/remove`	Player

Table 4: Mapping of Green User Stories (Additional Features) to API Endpoints.

4 Game Rules

This section outlines the logic and rules implemented for the digital adaptation of **La Escoba**. These rules represent the specific design decisions taken to standardize the gameplay for all users on the platform.

4.1 General Objective

The primary goal of the game is to capture cards from the central table by matching them with a card from the player's hand. The match is won by the player who accumulates the highest score based on the cards captured and specific achievements completed during the game.

4.2 Card Values Decision

To facilitate the core mechanic of summing exactly to **15**, the traditional values of the Spanish Deck were adapted for the digital engine.

- **Numeric Cards (1-7):** Retain their face value.
- **Face Cards (Sota, Caballo, Rey):** Were assigned the values **8, 9, and 10** respectively.
- **Decision Rationale:** This mapping ensures that every single card in the deck can form a valid combination to reach 15 (e.g., King [10] + 5 = 15).

4.3 Gameplay Mechanics

The game session follows a strict automated flow enforced by the server:

1. **Dealing:** The system deals 3 cards to each player and places 4 cards face-up on the table.
2. **The Turn:** On their turn, a player selects exactly one card from their hand to play.
3. **Capture Logic:**
    - The system automatically checks if the played card + any combination of cards on the table equals **15**.
    - If a match exists, the player captures the played card and the table cards.
    - If no match exists, the played card remains on the table (known as a "discard").
4. **Escoba:** If a capture clears the table completely, it counts as an "Escoba", awarding an immediate bonus point (unless it occurs in the very last hand).
5. **Refilling:** When players exhaust their hands, the system deals 3 new cards to each. This repeats until the deck is empty.
6. **Last Capture Rule:** At the end of the game, any cards remaining on the table are automatically awarded to the player who made the last valid capture.

4.4 Scoring System

The winner is determined by calculating the following points based on the captured "loot":

Criteria	Condition	Points
Escobas	Each time the player cleared the table.	+1 each
Cards (Cartas)	Capturing more than 20 cards total.	+1
Oros (Coins)	Capturing more than 5 cards of the *Oros* suit.	+1
Settebello	Capturing the 7 of Oros (Seven of Coins).	+1
Seventy (Setenta)	(Optional variant) Highest prime value in each suit.	+1

Table 5: Points distribution logic implemented in the Match Service.

5 Game Flow

This section details the specific API sequence required to simulate a complete game loop between two players (**Mario** and **Luigi**) using Postman. This flow demonstrates authentication, match creation, turn-based card play, and state synchronization.

Prerequisites:

- Both players are registered and logged in via `POST /auth/login`.
- The client possesses the JWT `access_token` for both users.
- All requests must include the header: `Authorization: Bearer <token>`.

5.1 Step 1: Match Creation

Mario initiates a match against Luigi. This process automatically deals cards and sets the initial state in Redis.

- **Player:** Mario (Using Mario's JWT)
- **Method:** `POST`
- **Endpoint:** `https://localhost:5000/matches`
- **Body (JSON):**

```
{
  "player1": "mario",
  "player2": "luigi"
}
```

- **Response:** Returns the `match_id` (e.g., `550e8400...`). **This ID must be saved** for all subsequent requests.

5.2 Step 2: Retrieving Initial State (Mario's Turn)

Mario checks his hand and the cards on the table before making his first move.

- **Player:** Mario (Using Mario's JWT)
- **Method:** `GET`
- **Endpoint:** `https://localhost:5000/matches/<match_id>?player=mario`
- **Note:** The `player` query parameter is crucial for the `Match Service` to identify the user and expose only Mario's hand, keeping Luigi's hand secret.
- **Expected State:** `turn` field should be "mario".

5.3 Step 3: Mario Plays a Card (Move 1)

Mario selects a card ID (e.g., `card_id: 10`) from his current hand and attempts to play it.

- **Player:** Mario
- **Method:** `POST`
- **Endpoint:** `https://localhost:5000/matches/<match_id>/play`
- **Body (JSON):**

```
{
  "player": "mario",
  "card_id": 10
}
```

- **Response:** The server validates the move, calculates captures (sum 15), updates the Redis state, and returns the result (e.g., `captured: [15, 20]`, `escoba: true`). The turn field is automatically updated to "luigi".

5.4 Step 4: Luigi Plays a Card (Move 2)

Luigi requests the game state to confirm the table configuration and then plays his card (e.g., `card_id: 5`).

- **Player:** Luigi (Must use Luigi's JWT)
- **State Check:** `GET https://localhost:5000/matches/<match_id>?player=luigi`
- **Play Method:** `POST https://localhost:5000/matches/<match_id>/play`
- **Body (JSON):**

```
{
  "player": "luigi",
  "card_id": 5
}
```

5.5 Step 5: Game Loop and Conclusion

Steps 2 through 4 are repeated until all cards are played.

- **Refilling Hands:** When both hands are empty, subsequent state checks will show new cards dealt from the deck.
- **Surrender (Optional):** Either player can terminate the match at any time using: `POST /matches/<match_id>/surrender` (Body: `{"player": "mario"}`).
- **Game Over:** The `Match Service` calculates the final scores and sends the immutable log to the `History Service`.

5.6 Step 6: Final Verification

Both players can query the immutable result and updated statistics.

- **Match Log:** `GET https://localhost:5000/history/match/<match_id>`
- **Leaderboard Check:** `GET https://localhost:5000/players/leaderboard/top`

6 Testing

The testing strategy was designed to ensure the reliability and security of the API Gateway and the core microservices. We utilized three distinct testing approaches, with all collections and scripts made executable in the `tests/` folder, as required.

6.1 Test Types and Coverage

- **Isolation Tests (Unit Tests):** Implemented using Postman Collections. These tests verify the business logic of individual microservices without relying on the network availability of others.
- **Integration Tests (API Gateway):** A Postman Collection dedicated to testing end-to-end flows (e.g., Register → Login → Create Match). These tests ensure the `API Gateway` correctly handles JWT token validation and traffic routing to the correct backend service.
- **Performance Tests (Load Testing):** Implemented using **Locust**. This verifies the system's capacity under high concurrent load, simulating users logging in and engaging in the most performance-intensive action: playing a card.

6.2 Particular Facts and Design Decisions

The microservices architecture necessitated specific strategies to achieve effective testing. The following points highlight key decisions made during the validation phase:

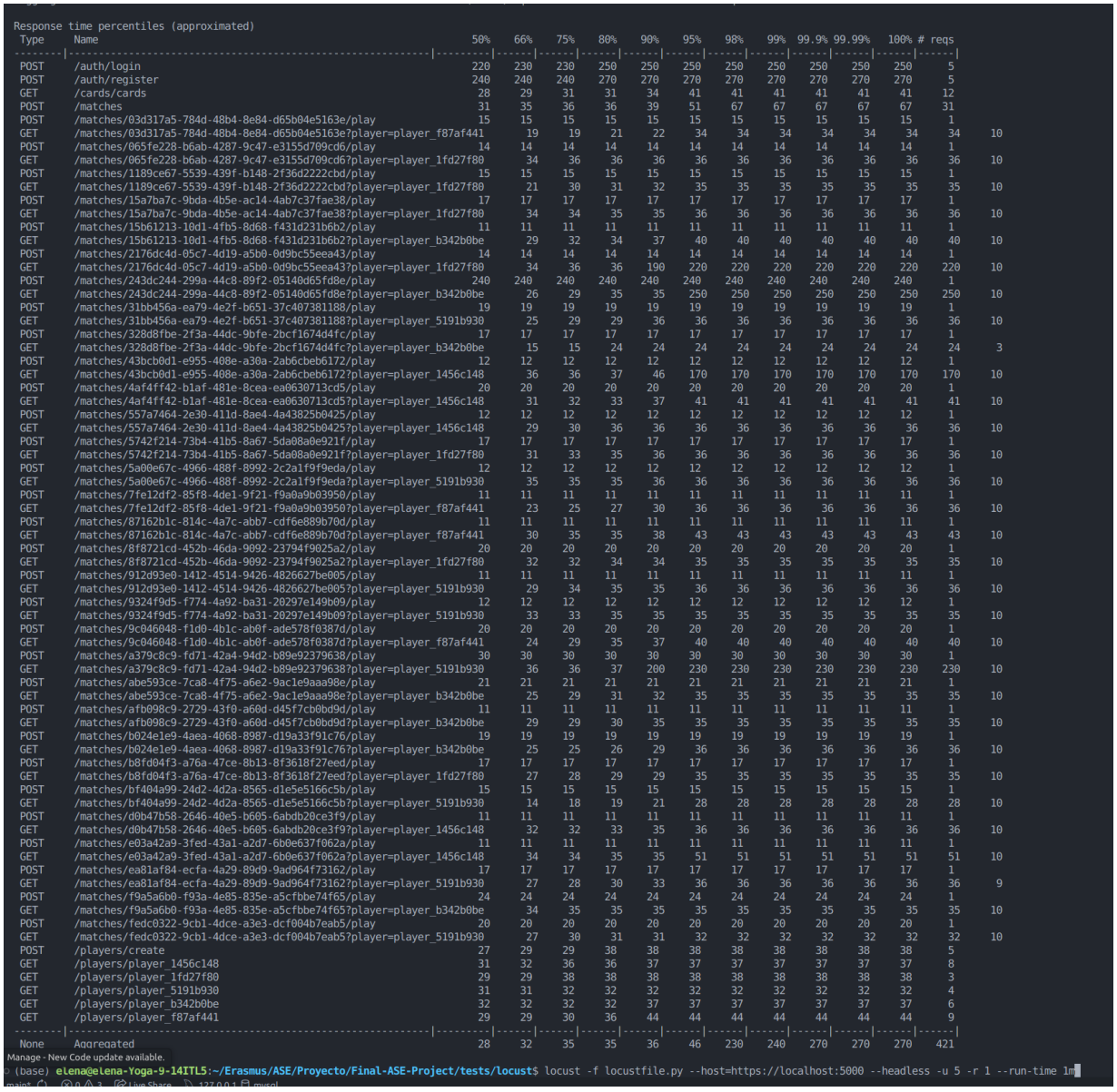
- **Service Isolation via Mocking:** The `Match Service` logic depends on the `Player Service` for stats updates. To test the game engine in strict isolation, we mocked external HTTP requests. For instance, calls to `/players/stats` were stubbed to return `200 OK` instantly, preventing network latency or database locks from affecting unit test results.
- **Dynamic Token Injection:** Since all gameplay endpoints are protected, hardcoding tokens is impossible due to the 24-hour expiration. Our Postman Integration collection utilizes a *Pre-request Script* that automatically executes a login, extracts the fresh `access_token`, and injects it into the environment variables for subsequent requests.
- **SSL Verification Bypass:** Because the development environment uses self-signed certificates for HTTPS, standard test runners would fail. We configured both Locust and the Python integration scripts with a custom SSL context (e.g., `verify=False`) to acknowledge the encryption without rejecting the untrusted local certificate authority.
- **State Cleanup Strategy:** To ensure tests are repeatable (idempotent), we implemented a teardown routine. After the performance tests, a script connects to Redis to flush the matchmaking queues, preventing "ghost matches" from polluting the next test run.
- **Edge Case Validation:** Beyond the "Happy Path" (a normal game), we specifically tested violation scenarios. For example, attempting to play a card ID that does not exist in the player's hand returns a `400 Bad Request`, and trying to play out of turn returns `403 Forbidden`.

7 Security – Data

This section details the measures taken to ensure data integrity and confidentiality, focusing on input sanitization mechanisms and data encryption strategies employed within the microservices architecture

7.1 Performance Test Results (Locust)

The following image shows the results of the load test performed using Locust. It displays the response time percentiles for each endpoint under a concurrent user load, confirming that the system maintains acceptable latency levels (under 300ms for the 95th percentile in most critical endpoints).



Response time percentiles (approximated)

Type	Name	50%	66%	75%	80%	90%	95%	98%	99%	99.9%	99.99%	100%	# reqs
POST	/auth/login	220	230	230	250	250	250	250	250	250	250	250	5
POST	/auth/register	240	240	240	270	270	270	270	270	270	270	270	5
GET	/cards/cards	28	29	31	31	34	41	41	41	41	41	41	12
POST	/matches	31	35	36	36	39	51	67	67	67	67	67	31
POST	/matches/03d317a5-784d-48b4-8e84-d65b04e5163e/play	15	15	15	15	15	15	15	15	15	15	15	1
GET	/matches/03d317a5-784d-48b4-8e84-d65b04e5163e?player=player_f87af441	19	19	21	22	34	34	34	34	34	34	34	10
POST	/matches/065fe228-b6ab-4287-9c47-e3155d709cd6/play	14	14	14	14	14	14	14	14	14	14	14	1
GET	/matches/065fe228-b6ab-4287-9c47-e3155d709cd6?player=player_1fd27f80	34	36	36	36	36	36	36	36	36	36	36	10
POST	/matches/1189ce67-5539-439f-b148-2f36d2222cbd/play	15	15	15	15	15	15	15	15	15	15	15	1
GET	/matches/1189ce67-5539-439f-b148-2f36d2222cbd?player=player_1fd27f80	21	30	31	32	35	35	35	35	35	35	35	10
POST	/matches/15a7ba7c-9bda-4b5e-ac14-4ab7c37fae38/play	17	17	17	17	17	17	17	17	17	17	17	1
GET	/matches/15a7ba7c-9bda-4b5e-ac14-4ab7c37fae38?player=player_1fd27f80	34	34	35	35	36	36	36	36	36	36	36	10
POST	/matches/15b61213-10d1-4fb5-8d68-f431d231b6b2/play	11	11	11	11	11	11	11	11	11	11	11	1
GET	/matches/15b61213-10d1-4fb5-8d68-f431d231b6b2?player=player_b342b0be	29	32	34	37	40	40	40	40	40	40	40	10
POST	/matches/2176dc4d-05c7-4d19-a5b0-0d9bc55eea43/play	14	14	14	14	14	14	14	14	14	14	14	1
GET	/matches/2176dc4d-05c7-4d19-a5b0-0d9bc55eea43?player=player_1fd27f80	34	36	36	190	220	220	220	220	220	220	220	10
POST	/matches/243dc244-299a-44c8-89f2-05140d65fd8e/play	240	240	240	240	240	240	240	240	240	240	240	1
GET	/matches/243dc244-299a-44c8-89f2-05140d65fd8e?player=player_b342b0be	26	29	35	35	250	250	250	250	250	250	250	10
POST	/matches/31bb456a-ea79-4e2f-b651-37c407381188/play	19	19	19	19	19	19	19	19	19	19	19	1
GET	/matches/31bb456a-ea79-4e2f-b651-37c407381188?player=player_5191b930	25	29	29	36	36	36	36	36	36	36	36	10
POST	/matches/328d8fbe-2f3a-44dc-9bfe-2bcf1674d4fc/play	17	17	17	17	17	17	17	17	17	17	17	1
GET	/matches/328d8fbe-2f3a-44dc-9bfe-2bcf1674d4fc?player=player_b342b0be	15	15	24	24	24	24	24	24	24	24	24	3
POST	/matches/43bcb0d1-e955-408e-a30a-2ab6cbeb6172/play	12	12	12	12	12	12	12	12	12	12	12	1
GET	/matches/43bcb0d1-e955-408e-a30a-2ab6cbeb6172?player=player_1456c148	36	36	37	46	170	170	170	170	170	170	170	10
POST	/matches/4af4ff42-b1af-481e-8cea-ea0630713cd5/play	20	20	20	20	20	20	20	20	20	20	20	1
GET	/matches/4af4ff42-b1af-481e-8cea-ea0630713cd5?player=player_1456c148	31	32	33	37	41	41	41	41	41	41	41	10
POST	/matches/557a7464-2e30-411d-8ae4-4a43825b0425/play	12	12	12	12	12	12	12	12	12	12	12	1
GET	/matches/557a7464-2e30-411d-8ae4-4a43825b0425?player=player_1456c148	29	30	36	36	36	36	36	36	36	36	36	10
POST	/matches/5742f214-73b4-41b5-8a67-5da08a0e921f/play	17	17	17	17	17	17	17	17	17	17	17	1
GET	/matches/5742f214-73b4-41b5-8a67-5da08a0e921f?player=player_1fd27f80	31	33	35	36	36	36	36	36	36	36	36	10
POST	/matches/5a00e67c-4966-488f-8992-2c2a1f9f9eda/play	12	12	12	12	12	12	12	12	12	12	12	1
GET	/matches/5a00e67c-4966-488f-8992-2c2a1f9f9eda?player=player_5191b930	35	35	35	36	36	36	36	36	36	36	36	10
POST	/matches/7fe12df2-85f8-4de1-9f21-f9a0a9b03950/play	11	11	11	11	11	11	11	11	11	11	11	1
GET	/matches/7fe12df2-85f8-4de1-9f21-f9a0a9b03950?player=player_f87af441	23	25	27	30	36	36	36	36	36	36	36	10
POST	/matches/87162b1c-814c-4a7c-abb7-cdf6e889b70d/play	11	11	11	11	11	11	11	11	11	11	11	1
GET	/matches/87162b1c-814c-4a7c-abb7-cdf6e889b70d?player=player_f87af441	30	35	35	38	43	43	43	43	43	43	43	10
POST	/matches/8f8721cd-452b-46da-9092-23794f9025a2/play	20	20	20	20	20	20	20	20	20	20	20	1
GET	/matches/8f8721cd-452b-46da-9092-23794f9025a2?player=player_1fd27f80	32	32	34	34	35	35	35	35	35	35	35	10
POST	/matches/912d93e0-1412-4514-9426-4826627be005/play	11	11	11	11	11	11	11	11	11	11	11	1
GET	/matches/912d93e0-1412-4514-9426-4826627be005?player=player_5191b930	29	34	35	35	36	36	36	36	36	36	36	10
POST	/matches/9324f9d5-f774-4a92-ba31-20297e149b09/play	12	12	12	12	12	12	12	12	12	12	12	1
GET	/matches/9324f9d5-f774-4a92-ba31-20297e149b09?player=player_5191b930	33	33	35	35	35	35	35	35	35	35	35	10
POST	/matches/9c046048-f1d0-4b1c-ab0f-ade578f0387d/play	20	20	20	20	20	20	20	20	20	20	20	1
GET	/matches/9c046048-f1d0-4b1c-ab0f-ade578f0387d?player=player_f87af441	24	29	35	37	40	40	40	40	40	40	40	10
POST	/matches/a379c8c9-fd71-42a4-94d2-b89e92379638/play	30	30	30	30	30	30	30	30	30	30	30	1
GET	/matches/a379c8c9-fd71-42a4-94d2-b89e92379638?player=player_5191b930	36	36	37	200	230	230	230	230	230	230	230	10
POST	/matches/abe593ce-7ca8-4f75-a6e2-9ac1e9aaa98e/play	21	21	21	21	21	21	21	21	21	21	21	1
GET	/matches/abe593ce-7ca8-4f75-a6e2-9ac1e9aaa98e?player=player_b342b0be	25	29	31	32	35	35	35	35	35	35	35	10
POST	/matches/afb098c9-2729-43f0-a60d-d45f7cb0bd9d/play	11	11	11	11	11	11	11	11	11	11	11	1
GET	/matches/afb098c9-2729-43f0-a60d-d45f7cb0bd9d?player=player_b342b0be	29	29	30	35	35	35	35	35	35	35	35	10
POST	/matches/b024e1e9-4aea-4068-8987-d19a33f91c76/play	19	19	19	19	19	19	19	19	19	19	19	1
GET	/matches/b024e1e9-4aea-4068-8987-d19a33f91c76?player=player_b342b0be	25	25	26	29	36	36	36	36	36	36	36	10
POST	/matches/b8fd04f3-a76a-47ce-8b13-8f3618f27eed/play	17	17	17	17	17	17	17	17	17	17	17	1
GET	/matches/b8fd04f3-a76a-47ce-8b13-8f3618f27eed?player=player_1fd27f80	27	28	29	29	35	35	35	35	35	35	35	10
POST	/matches/bf404a99-24d2-4d2a-8565-d1e5e5166c5b/play	15	15	15	15	15	15	15	15	15	15	15	1
GET	/matches/bf404a99-24d2-4d2a-8565-d1e5e5166c5b?player=player_5191b930	14	18	19	21	28	28	28	28	28	28	28	10
POST	/matches/d0b47b58-2646-40e5-b605-6abdb20ce3f9/play	11	11	11	11	11	11	11	11	11	11	11	1
GET	/matches/d0b47b58-2646-40e5-b605-6abdb20ce3f9?player=player_1456c148	32	32	33	35	36	36	36	36	36	36	36	10
POST	/matches/e03a42a9-3fed-43a1-a2d7-6b0e637f062a/play	11	11	11	11	11	11	11	11	11	11	11	1
GET	/matches/e03a42a9-3fed-43a1-a2d7-6b0e637f062a?player=player_1456c148	34	34	35	35	51	51	51	51	51	51	51	10
POST	/matches/ea81af84-ecfa-4a29-89d9-9ad964f73162/play	17	17	17	17	17	17	17	17	17	17	17	1
GET	/matches/ea81af84-ecfa-4a29-89d9-9ad964f73162?player=player_5191b930	27	28	30	33	36	36	36	36	36	36	36	9
POST	/matches/f9a5a6b0-f93a-4e85-835e-a5cfbbe74f65/play	24	24	24	24	24	24	24	24	24	24	24	1
GET	/matches/f9a5a6b0-f93a-4e85-835e-a5cfbbe74f65?player=player_b342b0be	34	35	35	35	35	35	35	35	35	35	35	10
POST	/matches/fedc0322-9cb1-4dce-a3e3-dcf004b7eab5/play	20	20	20	20	20	20	20	20	20	20	20	1
GET	/matches/fedc0322-9cb1-4dce-a3e3-dcf004b7eab5?player=player_5191b930	27	30	31	31	32	32	32	32	32	32	32	10
POST	/players/create	27	29	29	38	38	38	38	38	38	38	38	5
GET	/players/player_1456c148	31	32	36	36	37	37	37	37	37	37	37	8
GET	/players/player_1fd27f80	29	29	38	38	38	38	38	38	38	38	38	3
GET	/players/player_5191b930	31	31	32	32	32	32	32	32	32	32	32	4
GET	/players/player_b342b0be	32	32	32	32	37	37	37	37	37	37	37	6
GET	/players/player_f87af441	29	29	30	36	44	44	44	44	44	44	44	9
None	Aggregated	28	32	35	35	36	46	230	240	270	270	270	421

Figure 3: Detailed response time percentiles generated by Locust during the 1-minute load test.

7.2 Input Sanitization

Per the project requirements, we enforce strict validation on user inputs to prevent injection attacks and ensure data quality. We selected the **User Password** during registration as the

primary example of this process.

- **Selected Input:** The `password` field provided in the JSON body of the `POST /auth/register` endpoint.
- **Representation:** This input represents the secret credential the user intends to use for authentication. It is critical data that must meet specific complexity standards before being processed.
- **Microservice:** `Auth Service`.
- **Sanitization & Validation Method:** The service implements a dedicated validator function (e.g., `validate_password_complexity`) before the data touches the database. The sanitization process ensures:
  1. **Length Check:** Must be between 8 and 20 characters.
  2. **Character Classes:** Must contain at least one uppercase letter, one number, and one special character.
  3. **SQL Injection Prevention:** The system uses SQLAlchemy ORM, which automatically parameterizes inputs, treating the password string strictly as literal data, effectively neutralizing SQL injection attempts.

  If the input fails these checks, the request is rejected immediately with a `400 Bad Request`, and the data is discarded.

7.3 Data Encrypted at Rest

To protect sensitive user information in the event of a database compromise, we ensure that critical credentials are never stored in plain text.

The primary data encrypted at rest within the system are User Passwords, which serve as the authentication proof for user accounts. These are stored in the PostgreSQL database (specifically in the `users` table, column `password_hash`). The encryption and decryption logic (hashing and verification) is handled exclusively at the application level by the Auth Service, ensuring that the database engine itself never sees the plain text credentials.

Implementation Details:

- **Encryption Strategy:** We use Hashing (one-way encryption) rather than reversible encryption.
- **Algorithm: bcrypt** with automatic salt generation.
- **Process:**
  - On Write (Register): The `Auth Service` receives the plain text password, hashes it using `bcrypt.hashpw()`, and stores only the resulting hash in PostgreSQL.
  - On Read (Login): The `Auth Service` retrieves the stored hash and uses `bcrypt.checkpw()` to verify the input. The plain text password is never saved or logged.

8 Security – Authorization and Authentication

This section describes the security architecture regarding user identity and access control. We selected a Token-Based Authentication scenario using JWT (JSON Web Tokens), implementing a centralized issuance strategy with Gateway-level enforcement.

8.1 Selected Scenario: Centralized Issuance, Gateway Validation

We utilize a Symmetric Key architecture . The `Auth Service` is the sole authority for generating tokens, while the `API Gateway` acts as the primary firewall, validating tokens before routing traffic to internal microservices.

Key Management and Storage:

- **Type:** Symmetric Key.
- **Storage:** The key is stored as an environment variable (`SECRET_KEY`) within the Docker containers. It is never hardcoded in the source files.
- **Usage:** The same key is injected into the `Auth Service` (to sign) and the `API Gateway` (to verify signature).

8.2 Token Validation Workflow

The following list details the steps taken for every protected request (e.g., `POST /matches`):

1. **Client Request:** The user sends an HTTP request with the header: `Authorization: Bearer <ey...token...>`
2. **Gateway Interception:** The API Gateway intercepts the request before it reaches the backend logic.
3. **Signature Verification:** The Gateway uses the shared `SECRET_KEY` to verify the digital signature. If the signature does not match the payload, the token is considered forged.
4. **Expiration Check:** The Gateway decodes the payload and checks the `exp` (Expiration Time) claim against the current server time.
5. **Decision:**
   - Valid: The Gateway forwards the request to the upstream service (e.g., Match Service), passing the user identity context.
   - Invalid: The Gateway immediately returns `401 Unauthorized`, terminating the flow.

8.3 Access Token Payload Format

The Access Token is a standard JWT containing the minimal necessary information to identify the user and manage session validity.

Claim	Value Type	Description
`sub`	Integer	**Subject.** The unique User ID (Database Primary Key). Used for internal relationships.
`name`	String	**Username.** The human-readable identifier, used for display and routing.
`iat`	Timestamp	**Issued At.** The exact time the token was created.
`exp`	Timestamp	**Expiration Time.** Set to exactly 24 hours after `iat`.

Table 6: Structure of the JWT Payload.

8.4 Handling Expired Access Tokens

The system enforces a strict expiration policy to minimize the attack window in case of token theft.

- **Detection:** The validation library (PyJWT) automatically raises an `ExpiredSignatureError` if the current timestamp is greater than the `exp` claim.
- **System Response:** The API Gateway catches this error and returns a generic **401 Unauthorized** response with the message `"Token expired"`.
- **Client Action:** Since we did not implement Refresh Tokens (to reduce complexity and attack surface for this MVP), the client (Frontend) redirects the user to the **Login Page**, forcing a re-authentication with credentials to obtain a new token.

9 Security - Analyses

This section details the security analysis performed on the codebase using automated tools to identify potential vulnerabilities and coding flaws.

9.1 Static Code Analysis (Bandit)

The project's Python codebase was audited using **Bandit** to identify common security issues. The scan focused on all microservices within the `services/` directory.

Execution Command:

```
bandit -r services/
```

```
(base) elena@elena-Yoga-9-14ITL5:~/Erasmus/ASE/Proyecto/Final-ASE-Project$ bandit -r services/
[main]  INFO    profile include tests: None
[main]  INFO    profile exclude tests: None
[main]  INFO    cli include tests: None
[main]  INFO    cli exclude tests: None
[main]  INFO    running on Python 3.12.3
Run started:2025-12-16 19:54:29.780466+00:00

Test results:
>> Issue: [B105:hardcoded_password_string] Possible hardcoded password: 'frontend_secret_key'
   Severity: Low   Confidence: Medium
   CWE: CWE-259 (https://cwe.mitre.org/data/definitions/259.html)
   More Info: https://bandit.readthedocs.io/en/1.9.2/plugins/b105_hardcoded_password_string.html
   Location: services/frontend/app.py:9:17
8       app = Flask(__name__)
9       app.secret_key = 'frontend_secret_key'
10      debug_mode = os.environ.get('FLASK_DEBUG', 'False').lower() == 'true'

--------------------------------------------------
>> Issue: [B105:hardcoded_password_string] Possible hardcoded password: 'Admin123!'
   Severity: Low   Confidence: Medium
   CWE: CWE-259 (https://cwe.mitre.org/data/definitions/259.html)
   More Info: https://bandit.readthedocs.io/en/1.9.2/plugins/b105_hardcoded_password_string.html
   Location: services/frontend/app.py:440:47
439             password = request.form.get('password')
440             if username == 'admin' and password == 'Admin123!':
441                 session['admin_logged_in'] = True

--------------------------------------------------

Code scanned:
        Total lines of code: 1323
        Total lines skipped (#nosec): 0

Run metrics:
        Total issues (by severity):
                Undefined: 0
                Low: 2
                Medium: 0
                High: 0
        Total issues (by confidence):
                Undefined: 0
                Low: 0
                Medium: 2
                High: 0
Files skipped (0):
```

Figure 4: Result of the Static Code Analysis performed by Bandit, showing identified issues in the frontend service.

9.1.1 Analysis of Identified Issues

The analysis identified specific security concerns that are documented for review:

- **Issue B105 (Hardcoded Password):** Bandit flagged the presence of hardcoded strings such as `'frontend_secret_key'` and `'Admin123!'` in `services/frontend/app.py`. These represent a "Low" severity risk with "Medium" confidence.

- **Metric Summary:** The scan covered 1,323 lines of code, identifying 2 low-severity issues related to hardcoded credentials, as shown in Figure 4.

9.1.2 Mitigation of Identified Critical Issues

To achieve a final result of zero issues, it was necessary to address and fix the following vulnerabilities identified during the initial scan, which primarily affected the secure deployment environment and communication:

- **High Severity (B201: `flask_debug_true`): Why:** Running Flask applications with `debug=True` (found in multiple services, including the `API Gateway`) is a critical security risk, as it exposes the Werkzeug debugger, allowing for arbitrary code execution. **How:** This was mitigated by ensuring that all services rely exclusively on the production server (`Gunicorn`) for execution (as defined in `docker-compose.yml`), and removing the redundant `app.run(debug=True, ...)` calls from the main execution block of each application.
- **High Severity (B501: `request_with_no_cert_validation`): Why:** Found in the `match-service` when communicating with `history-service` and `player-service`. Using `verify=False` in internal `requests.post()` calls disables SSL certificate checks, exposing inter-service communication to man-in-the-middle (MITM) attacks, which violates the HTTPS-only requirement. **How:** The `verify=False` flag was replaced with an explicit path to the self-signed certificate (`verify='/app/certs/cert.pem'`) in the `match-service`. This forces the service to validate the self-signed certificate, ensuring encrypted communication and integrity verification.
- **Medium Severity (B104: `hardcoded_bind_all_interfaces`): Why:** Warning related to binding applications to all interfaces (`host='0.0.0.0'`), which is bad practice when not necessary. **How:** This issue was resolved implicitly by eliminating the local execution code (the `if __name__ == '__main__':` block) used for fixing B201, as Gunicorn handles the correct internal binding.

9.1.3 Mitigation of Low Severity Issues

- **Low Severity (B105: Hardcoded Secrets): Why:** The session secret key (`frontend_secret_key`) and the default Admin password (`Admin123!`) were hardcoded in the `frontend/app.py`. **How:** All hardcoded secrets were moved to be loaded via `os.environ.get(...)`. This requires defining these values exclusively in the `docker-compose.yml` environment block, isolating secrets from the source code.
- **Low Severity (B110: Try, Except, Pass): Why:** Several instances of `except: pass` were used in the `frontend` service, which masked critical errors (e.g., network failure, JSON decode error) when communicating with the `API Gateway`. **How:** The blind exception blocks were replaced by explicitly capturing specific exceptions from the `requests` library (`RequestException`, `JSONDecodeError`). This ensures that unexpected errors are handled gracefully and logged, rather than being silenced.

9.2 Docker Image Vulnerability Scanning

The developed Docker images (based on `python:3.10-slim`) were scanned using **Trivy** (an open-source vulnerability scanner) to ensure compliance with the mandatory requirement: images must be free from Critical and High-severity vulnerabilities.

The following table summarizes the final scan results after successfully patching the `gunicorn` dependency (upgrading from `21.2.0` to `22.0.0`) across all services, which mitigated the High-severity HTTP Request Smuggling issues (CVE-2024-1135 and CVE-2024-6827).

Table 7: Final Vulnerability Scan Results for Docker Microservice Images

Image Name	Critical	High	Medium	Low	Status
`auth-service`	0	0	0	0	**PASS**
`player-service`	0	0	0	0	**PASS**
`match-service`	0	0	0	0	**PASS**
`history-service`	0	0	0	0	**PASS**
`cards-service`	0	0	0	0	**PASS**
`api-gateway`	0	0	0	0	**PASS**
`frontend-service`	0	0	0	0	**PASS**

Figure 5: Example of a clean Trivy scan result confirming zero Critical or High vulnerabilities.

As shown in the Trivy scan results, the mitigation strategy of forcing `gunicorn>=22.0.0` and performing a clean build without cache resulted in the complete elimination of all High-severity vulnerabilities. All microservice images now report zero High or Critical issues, ensuring a secure production environment.

10 Security – Threat Model

Following the methodology from the Security Assessment Lab, we performed a threat modeling exercise to identify potential security risks in the "La Escoba" architecture. We analyzed the assets, attack surfaces, and trust boundaries before classifying the threats using the STRIDE model.

10.1 Identify Assets

These are the valuable items in our system that we need to protect:

- **A1 - User Data (PII):** Usernames, email addresses, and password hashes stored in the `Auth Database`.
- **A2 - Game Integrity:** The active state of a match (who has which cards, current score) stored in **Redis**. If this is modified, the game is broken.
- **A3 - Session Tokens:** The JWT `access_token` used to authenticate users[cite: 28].
- **A4 - Match History:** The permanent record of who won and the moves played, stored in the `History Database`.
- **A5 - Service Availability:** The ability of the platform to host matches without crashing.

10.2 Identify Attack Surface

These are the entry points where an attacker could try to enter the system:

- **Public API Endpoints:** The exposed routes on the API Gateway (Port 5000), specifically:
  - `/auth/login` and `/register` (Input validation risks).
  - `/matches/{id}/play` (Game logic risks).
- **Input Fields:** JSON payloads sent by the client (e.g., trying to play a card ID that the user doesn't have).

10.3 Identify Trust Boundaries

We identified the lines where data moves between different levels of trust:

- **TB1 (Internet Boundary):** Between the **Client (Frontend)** and the **API Gateway**. The client is untrusted (Zone 0), and the Gateway is the first line of defense.
- **TB2 (Service Boundary):** Between the **API Gateway** and the internal **Microservices**. Once inside, traffic is considered "internal" but still requires identity verification via JWT.
- **TB3 (Data Boundary):** Between the **Microservices** and the **Databases** (PostgreSQL/Redis).

10.4 STRIDE Analysis

We classified the potential threats using the STRIDE categories:

Category	Threat Example	Target Asset
S - Spoofing	An attacker steals a valid JWT token and impersonates "Mario" to play turns for him.	A3 (Session)
T - Tampering	A user tries to modify the game state in Redis to give themselves extra points or better cards.	A2 (Integrity)
R - Repudiation	A player loses a match and claims "I never played that card," preventing accountability.	A4 (History)
I - Info Disclosure	A user changes the URL ID (`/matches/5`) to peek at the cards of Match 6 (IDOR).	A2 (Integrity)
D - DoS	A bot creates thousands of matches in one second to fill up the Redis memory and crash the server.	A5 (Availability)
E - Elevation	A regular user tries to access admin-only endpoints to view all user profiles.	A1 (User Data)

10.5 Threat Mitigation Strategies

Based on the identified threats, we implemented the following controls:

- **Spoofing Mitigation:** We enforce HTTPS encryption to prevent token theft over the network and verify the JWT Signature at the API Gateway level on every request.
- **Tampering Mitigation:** The Match Service validates every move server-side. It checks if the card is actually in the user's hand before updating Redis. We never trust the client's claim.
- **Repudiation Mitigation:** The History Service writes immutable logs to PostgreSQL immediately after a match ends, ensuring a permanent record.
- **Info Disclosure Mitigation:** We implemented logic checks. When a user requests `GET /matches/{id}`, the service checks the `player` parameter and only returns the hand belonging to that specific user.
- **DoS Mitigation:** We use Redis TTL (Time To Live) to automatically delete abandoned matches after a set time, freeing up memory.

11 Use of Generative AI

In this section, we explain how we used Generative AI tools (like ChatGPT and Gemini) to help us build the "La Escoba" project. We used them mainly as a "virtual assistant" to write code faster and solve specific problems we didn't know how to fix.

11.1 AI Systems Used

We mainly used ChatGPT and Google Gemini. We used them like a search engine or a teammate: we asked questions, and they suggested code or solutions.

11.2 How AI Supported Us

Building a microservices project is complex. The AI helped us in these specific areas:

- **Setting up the project:** Writing the initial files for Flask and the long `docker-compose.yml` file takes a lot of time. The AI wrote the base code for us, saving us hours of typing.
- **Help with LaTeX:** When the images and tables moved to the wrong places (floating issues), the AI taught us how to use the `float` package to fix them. It acted as a teacher for the report formatting.
- **Creating Test Data:** We needed examples of JSON data to test the Match Service. The AI generated many examples for us to use in Postman.
- **Finding Libraries:** It recommended which Python libraries to use, for example, suggesting `PyJWT` for handling the security tokens.

11.3 Advantages

- **Speed:** We could write the basic code for the services (endpoints, database models) much faster.
- **Fixing Errors:** When Docker gave us difficult errors that we didn't understand, the AI explained what was wrong and how to connect the containers correctly.
- **Learning:** It helped us learn how to use tools we didn't know well, like Redis or advanced LaTeX features.

11.4 Disadvantages and Limitations

Even though it was helpful, the AI made mistakes, so we had to be careful:

- **Old Code:** Sometimes the AI gave us Python code that was too old and didn't work anymore. We had to check the documentation to fix it.
- **Confused Rules:** The AI didn't understand the specific rules of "La Escoba". It often tried to use rules from other card games. We had to write the scoring logic entirely by ourselves.
- **Making things up:** Sometimes it suggested functions that don't actually exist in Python. We had to test everything to make sure it was real.

Conclusion: Generative AI was a great helper for setting up the project and fixing bugs, especially with LaTeX and Docker. However, the real engineering work, the game rules, and the final decisions were done by the team.

12 Additional Features

In this section, we describe the features implemented beyond the mandatory Red and Yellow user stories. These features aim to provide a more complete social experience and better administrative control over the microservices ecosystem.

12.1 Feature 1: Admin Dashboard & Monitoring

- **What is this feature?** An exclusive web interface (`admin_dashboard.html`) accessible only to users with the "admin" role. It provides a real-time overview of the system's state.
- **Why is it useful?** In a microservices architecture, it is difficult to know if all services are synchronized. This dashboard allows us to monitor active matches, see total registered players, and verify system health without manual database queries.
- **How is it implemented?** We extended the `auth-service` to include a `role` claim in the JWT. The `api-gateway` validates this role. The dashboard performs server-side requests to other services to aggregate the data.
- **How is it used?** The administrator logs in via `/admin/login`. Once authenticated, they are redirected to a panel where they can see the list of all ongoing matches and system statistics.

12.2 Feature 2: Friend System & Social Interaction

- **What is this feature?** A social module that allows players to search for other users and add them to a personal "Friends List".
- **Why is it useful?** It fulfills the "Green" requirements related to social features. It enhances user retention by allowing players to keep track of their acquaintances within the game.
- **How is it implemented?** Implemented within the `player-service`. We used a Self-Referential Many-to-Many relationship in the database. We added specific endpoints (`/friends/add` and `/friends/list`) that handle these associations.
- **How is it used?** In the "Friends" section of the frontend, users can type a username to send a request and view the list of their current friends and their global ranking.

12.3 Feature 3: Waiting Room (Matchmaking Logic)

- **What is this feature?** An intermediate state between the dashboard and the game where the system searches for an available opponent.
- **Why is it useful?** It prevents the creation of empty or "orphan" matches. It ensures that the `match-service` only allocates resources when a pair of players is ready to start.
- **How is it implemented?** We implemented a polling mechanism. When a player clicks "Play", they are put in a `WAITING` state in the database. A background check in the `match-service` pairs two players in this state and generates a unique `match_id` for them.

- **How is it used?** The user enters the `waiting.html` screen. The frontend pings the API until an opponent is found, at which point the game UI (`game.html`) is automatically loaded.

Note: All additional features follow the same security patterns as the mandatory ones, requiring valid JWT authentication and HTTPS communication.